

CAPSTONE - I PROJECT REPORT

DESIGN & IMPLEMENTATION OF 16-Bit WALLACE TREE MULTIPLIER

A 5-Stage Pipelined RTL Implementation with Wallace-Tree Partial Product Reduction and Carry-Lookahead Final Addition

Name	Irukuvajula Dhanush
University	JAIN University, Bengaluru
Application. No	23152307
Course	VLSI Design
Design Language	Verilog HDL
Simulation Tool	Icarus Verilog + GTKWave

Table of Contents

1. Introduction	4
2. Project Objectives.....	5
3. System Overview.....	6
3.1 Top-Level Interface.....	6
3.2 Five-Stage Pipeline Structure.....	6
3.3 Module Hierarchy.....	7
4. Architecture and Design Details.....	9
4.1 Stage 1 — Partial Product Generation.....	9
4.2 Stages 2–4 — Wallace Tree Reduction.....	9
4.3 Stage 5 — 32-bit Carry Lookahead Adder.....	13
4.4 Pipeline Register Placement.....	14
5. Pipelining and Throughput Analysis.....	16
5.1 Latency.....	16
5.2 Throughput — Full Pipelining.....	16
5.3 Pipelined vs. Non-Pipelined Comparison.....	17
6. Verilog Implementation Summary.....	19
6.1 Design Style Notes.....	19
7. Verification Methodology.....	20
7.1 Self-Checking Testbench Design.....	20
7.2 Test Plan.....	20
7.3 Verification Flow Diagram.....	21
8. Simulation and Test Results.....	22
8.1 Simulation Header.....	22
8.2 Directed Corner-Case Results.....	22
8.3 Random Vector Results (Sample).....	22
8.4 Final Test Summary.....	22
9. Waveform Analysis (GTKWave).....	24
9.1 Observed Pipeline Behavior.....	24
9.2 Internal Wallace-Tree Signals.....	24
9.3 How to Reproduce.....	24
10. Synthesizability.....	25
11. Performance Discussion.....	26
11.1 Why Wallace Tree Over Sequential Multiplication.....	26
11.2 Why Pipeline the Reduction Tree.....	26
11.3 Why Carry Lookahead for the Final Add.....	26
11.4 Resource / Complexity Summary.....	26

12. Conclusion	27
13. Result Waveforms from GTKWave Simulation	28
13.1 Early Simulation Window (0–800 ns)	28
13.2 Zoomed Detail (300–600 ns)	29
13.3 Full Random-Test Run (0–1.6 μ s, Zoomed Out)	30
14. Appendices	31
Appendix A — Project File Inventory	31
Appendix B — Full Adder / Half Adder Truth Tables	31
Appendix C — Reproducing the Simulation	32

1. Introduction

This report documents the design, implementation, and verification of a 16-bit $\times$ 16-bit unsigned pipelined multiplier built around a Wallace Tree partial-product reduction network and a 32-bit Carry Lookahead Adder (CLA). The design is fully synchronous, organized into five registered pipeline stages, and was implemented from scratch in structural/dataflow Verilog using only primitive half-adder and full-adder building blocks for the reduction logic.

The motivation for this architecture is performance: a naive sequential (shift-and-add) multiplier needs on the order of 16 clock cycles to multiply two 16-bit operands, whereas a Wallace Tree reduces the sixteen 16-bit partial product rows down to two rows in only six reduction levels using parallel 3:2 and 2:2 compressors, after which a single fast adder produces the final product. By additionally pipelining that reduction tree across clock-edge boundaries, the design achieves a fixed 4-cycle latency while accepting one new pair of operands on every single clock cycle once the pipeline is full — i.e., one multiplication result per cycle in steady state.

The design was verified using a self-checking Verilog testbench that compares every output against a behavioral reference model (Verilog's native “*” operator, used only in the testbench, never in the design under test). The testbench applied 12 directed corner-case vectors (covering 0 $\times$ 0, max $\times$ max, and other boundary conditions) followed by 1,000 back-to-back pseudo-random vectors, for a grand total of 1,012 checked results.

All **1,012 of 1,012 results passed** — a 100% pass rate — with zero mismatches against the reference model, confirming both the functional correctness of the Wallace reduction network and the correct latency/throughput behavior of the pipeline.

All design source files, the self-checking testbench, the simulation log, and the GTKWave waveform dump (.vcd) referenced throughout this report are included as part of the project deliverables.

2. Project Objectives

The capstone project set out to design, implement, and verify a high-throughput unsigned multiplier suitable for use as a building block in a larger datapath (e.g., a DSP or ALU pipeline). The specific objectives were:

1. Implement a structural 16-bit $\times$ 16-bit unsigned multiplier using Wallace Tree partial-product reduction rather than a sequential shift-and-add approach.
2. Pipeline the reduction tree across multiple clock-edge-aligned register stages so the design can accept a new operand pair every clock cycle (full throughput pipelining).
3. Use only fundamental combinational primitives (half adders and full adders) to build every reduction level, demonstrating bottom-up structural design rather than relying on the '*' operator or vendor IP.
4. Use a Carry Lookahead Adder, rather than a ripple-carry adder, for the final 32-bit addition stage, to avoid reintroducing a long carry-propagation bottleneck after the fast Wallace reduction.
5. Build a self-checking testbench capable of verifying pipeline latency, applying both directed corner-case and randomized test vectors, and reporting a clear pass/fail summary.
6. Generate a waveform (.vcd) for visual timing verification in GTKWave, and produce complete supporting documentation (this report, architecture diagrams, and run instructions).

3. System Overview

At the top level, the design exposes a simple synchronous interface: two 16-bit operands, a valid-in handshake bit, a clock, and an active-low synchronous reset, and produces a 32-bit product together with a valid-out bit on the output side.

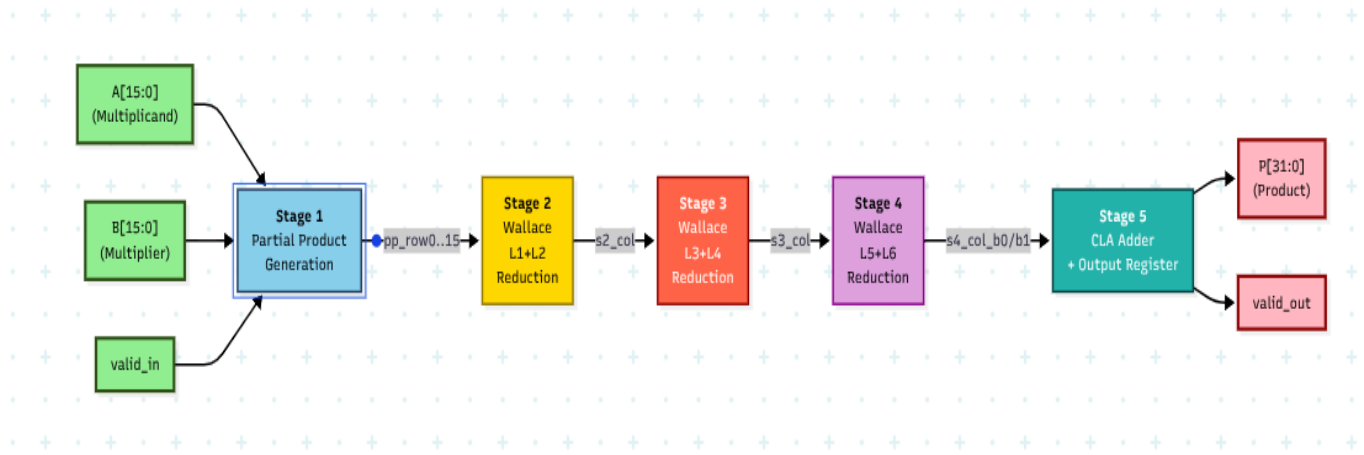


Figure 3.0 — High-level block diagram: Stage 1 (Partial Product Generation) through Stage 5 (CLA Adder + Output Register), showing the $A[15:0]$, $B[15:0]$, and $valid_in$ inputs flowing to the $P[31:0]$ and $valid_out$ outputs

This diagram mirrors the actual signal names used in the Verilog source: `pp_row0..15` is the 16-row partial-product bus leaving Stage 1, `s2_col` / `s3_col` are the intermediate Wallace-reduction buses leaving Stages 2 and 3, and `s4_col_b0/b1` are the final two 32-bit rows leaving Stage 4 that Stage 5's `cla_32bit` module adds together.

3.1 Top-Level Interface

Signal	Direction	Width	Description
<code>clk</code>	Input	1	System clock; design is purely synchronous, frequency-agnostic
<code>rst_n</code>	Input	1	Active-low synchronous reset; clears all pipeline registers
<code>valid_in</code>	Input	1	Asserted when A and B are valid on the current cycle
<code>A</code>	Input	16	Multiplicand
<code>B</code>	Input	16	Multiplier
<code>P</code>	Output	32	Registered final product, $P = A \times B$
<code>valid_out</code>	Output	1	Asserted exactly 4 cycles after a corresponding <code>valid_in</code> , marking P as valid

Table 3.1 — Top-level port list of `pipelined_wallace_mult_16x16`

3.2 Five-Stage Pipeline Structure

The module `pipelined_wallace_mult_16x16` is the top-level wrapper that instantiates each of the five functional stages below and places a bank of pipeline registers between every pair of adjacent stages. The `valid_in` bit is itself pipelined alongside the data, so `valid_out` automatically tracks the correct latency without any separate counter or state machine.

Stage	Module	Function
1	<code>partial_product_gen</code>	Generates all 256 raw partial-product bits via bitwise AND of A against each bit of B
2	<code>wallace_stage1</code>	Wallace reduction Levels 1 + 2 (first two layers of 3:2 / 2:2 compression)
3	<code>wallace_stage2</code>	Wallace reduction Levels 3 + 4 (continued column compression)
4	<code>wallace_stage3</code>	Wallace reduction Levels 5 + 6 (final compression down to exactly two 32-bit rows)
5	<code>cla_32bit</code>	32-bit Carry Lookahead Adder sums the final two rows to form P[31:0]; result is registered to form the output

Table 3.2 — Pipeline stage breakdown

Because there are five register boundaries along the datapath (the four inter-stage boundaries plus the final output register), a result for an operand pair applied at cycle T becomes valid at the output on cycle T+4 — a fixed latency of 4 clock cycles, confirmed directly by simulation and used by the testbench scoreboard described in Section 7.

3.3 Module Hierarchy

The instantiation hierarchy, from the testbench down to the bit-level adder primitives, is shown below.

```

tb_pipelined_wallace_mult_16x16 (self-checking testbench, instantiates DUT)
├── pipelined_wallace_mult_16x16 (TOP-LEVEL design under test)
│   ├── partial_product_gen (Stage 1: 256 AND-gate partial products)
│   ├── wallace_stage1 (Stage 2: Wallace Levels 1-2)
│   │   ├── full_adder (x124)
│   │   └── half_adder (x21)
│   ├── wallace_stage2 (Stage 3: Wallace Levels 3-4)
│   │   ├── full_adder (x53)
│   │   └── half_adder (x23)
│   ├── wallace_stage3 (Stage 4: Wallace Levels 5-6)
│   │   ├── full_adder (x19)
│   │   └── half_adder (x35)
│   └── cla_32bit (Stage 5: 32-bit Carry Lookahead Adder)
│       └── 8 x 4-bit lookahead blocks

```

Figure 3.1 — Module instantiation hierarchy with adder primitive counts per stage

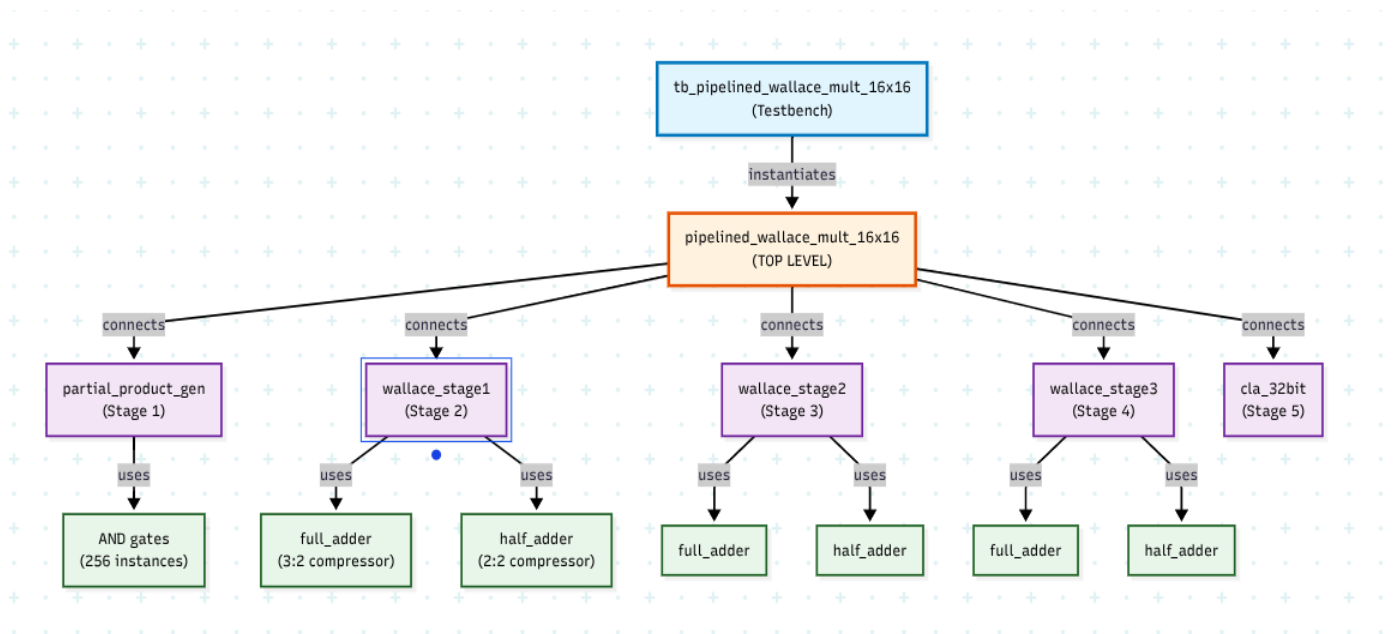


Figure 3.2 — Module instantiation diagram: `tb_pipelined_wallace_mult_16x16` instantiates the top-level design, which connects `partial_product_gen`, `wallace_stage1`, `wallace_stage2`, `wallace_stage3`, and `cla_32bit`, each of which uses `full_adder` / `half_adder` primitives (or, for Stage 1, 256 AND gates)

This diagram makes explicit a structural design rule followed throughout the project: every stage above the AND-gate level is built exclusively from the two compressor primitives defined in `full_adder.v` and `half_adder.v`. No stage ever instantiates a wider arithmetic primitive (such as a multi-bit adder IP block) for its internal reduction logic — only `cla_32bit`, the dedicated final-addition stage, uses anything beyond single-bit half/full adders, and even that module builds its lookahead logic from per-bit generate/propagate signals rather than a black-box adder.

4. Architecture and Design Details

4.1 Stage 1 — Partial Product Generation

The `partial_product_gen` module forms the sixteen raw partial-product rows of the multiplication. For row i , every bit of A is ANDed with the single bit $B[i]$:

```
pp_rowI = A & {16{B[I]}}; // row I is A if B[I]=1, else all-zero
```

This single bitwise-AND-with-replicated-mask expression computes all 16 AND gates for a row in one line, and is logically identical to forming $pp[i][j] = A[j] \cdot B[i]$ for every column j — exactly the partial product term used in pencil-and-paper long multiplication. Across all sixteen rows, this stage produces all 256 partial-product bits (16×16) needed for the reduction tree. Each row's bits implicitly belong to a column range starting at column i (row i is logically shifted left by i bit positions, although the shifting is realized simply by how Stage 2 reads bit j of row i as belonging to column $i+j$, with no explicit shift hardware required).

4.2 Stages 2–4 — Wallace Tree Reduction

A Wallace Tree compresses many partial-product bits that land in the same binary column into a much smaller number of rows, using two basic compressor primitives:

- Half Adder (2:2 compressor) — takes 2 bits in a column, produces 1 sum bit (stays in the same column) and 1 carry bit (moves up one column). $sum = a \oplus b$, $cout = a \cdot b$.
- Full Adder (3:2 compressor) — takes 3 bits in a column, produces 1 sum bit (same column) and 1 carry bit (next column up). $sum = a \oplus b \oplus cin$, $cout = (a \cdot b) + (b \cdot cin) + (a \cdot cin)$.

Because each Full Adder reduces three bits to two outputs in a single logic level, columns with many partial-product bits are reduced far faster than a chain of ripple-carry additions would allow — this is the central performance advantage of the Wallace Tree over naive accumulation. The reduction proceeds over six total levels, split across three pipeline stages for timing balance:

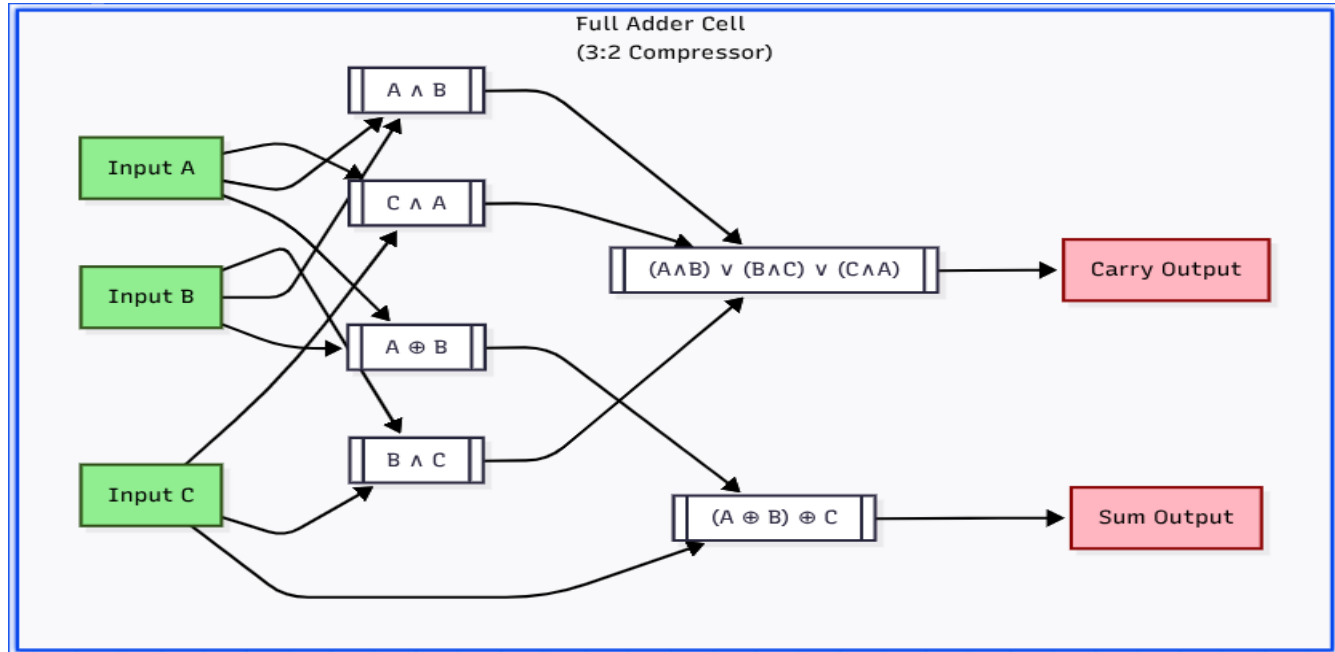


Figure 4.1 — Internal logic of the `full_adder` (3:2 compressor) cell: $Sum = (A \oplus B) \oplus C$, $Carry = (A \wedge B) \vee (B \wedge C) \vee (C \wedge A)$, exactly matching the two assign statements in `full_adder.v`

This is a direct visualization of the two combinational equations in `full_adder.v` — `assign sum = a ^ b ^ cin;` and `assign cout = (a & b) | (b & cin) | (a & cin);` — decomposed into individual AND/XOR/OR gates. Every one of the 196 `full_adder` instances used across `wallace_stage1`, `wallace_stage2`, and `wallace_stage3` is a copy of exactly this same small gate network, wired to a different triple of partial-product (or intermediate sum/carry) bits depending on its column position.

The `half_adder` cell used 79 times across the same three stages is the 2-input subset of this same logic: $sum = a \oplus b$ and $cout = a \cdot b$, used whenever a column has exactly 2 bits left over after grouping the rest into 3-bit Full Adder triples.

Pipeline Stage	Wallace Levels	Full Adders	Half Adders	Purpose
wallace_stage1 (Stage 2)	Level 1 + 2	124	21	First major collapse of the 16-row partial-product matrix
wallace_stage2 (Stage 3)	Level 3 + 4	53	23	Continued column-by-column compression
wallace_stage3 (Stage 4)	Level 5 + 6	19	35	Final reduction down to exactly two 32-bit rows

Table 4.1 — Adder primitive usage per Wallace reduction stage (196 full adders + 79 half adders = 275 adder primitives total)

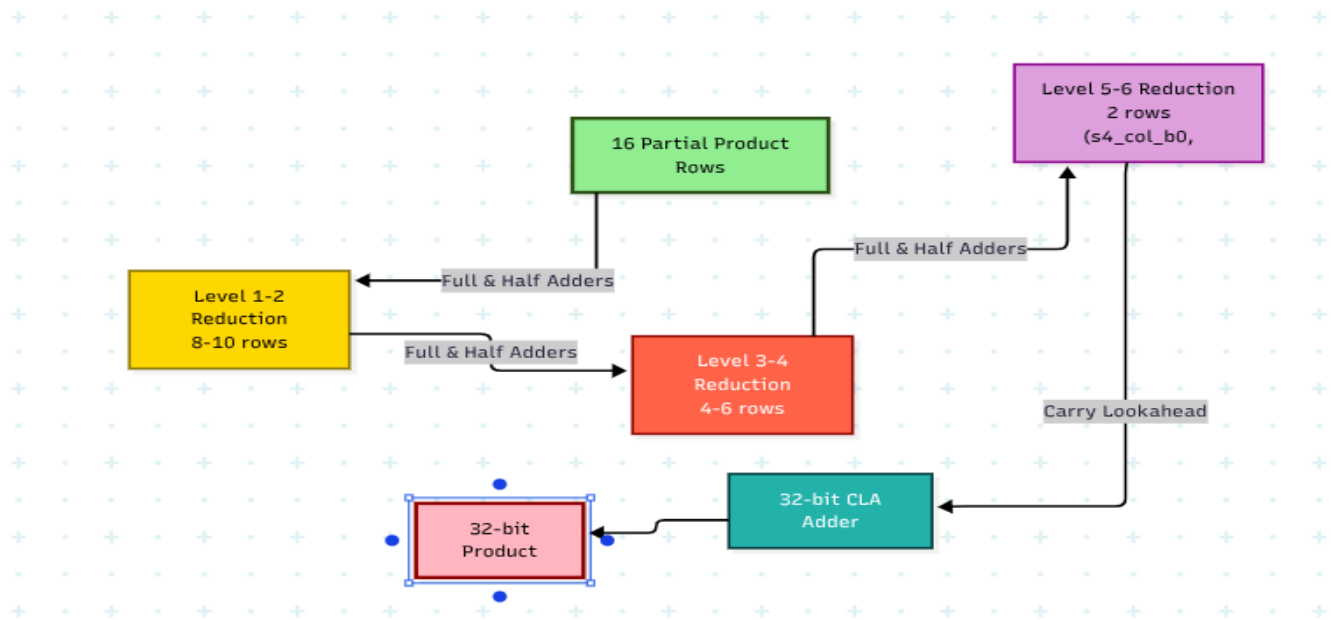


Figure 4.2 — Wallace Tree reduction flow: the 16 partial-product rows are reduced by Level 1-2 (8-10 rows), then Level 3-4 (4-6 rows), then Level 5-6 (down to exactly 2 rows: `s4_col_b0` and `s4_col_b1`), and finally summed by the 32-bit CLA

This figure summarizes the column-height reduction that the design's signal names directly encode. The `s2_col*` signals emerging from `wallace_stage1` hold between 8 and 10 rows of bits per column (visible in the source as suffixes like `s2_col12_b0` through `s2_col12_b5`, i.e. column 12 still has 6 bits after Level 1-2). The `s3_col*` signals emerging from `wallace_stage2` are narrower still (typically 4-6 bits per column), and by the time `wallace_stage3` finishes Level 5-6, every column has been collapsed to at most 2 bits — represented in the RTL as the paired `s4_col*_b0` / `s4_col*_b1` signals that form the two final rows handed to `cla_32bit`.

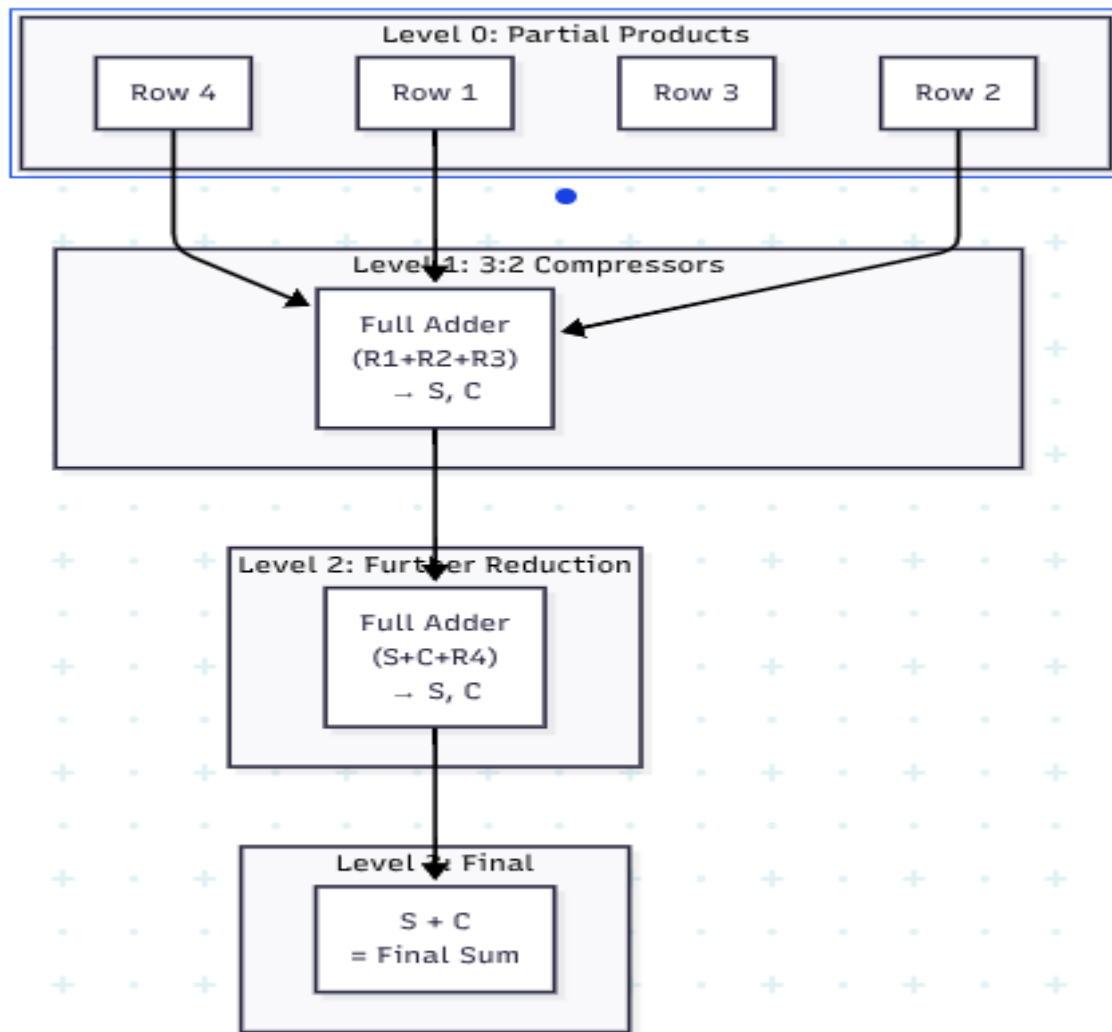


Figure 4.3 — Worked example of a 3-level column reduction: four partial-product rows (Row 1–Row 4) are reduced first by a Full Adder combining Row1+Row2+Row3 into a Sum/Carry pair, then a second Full Adder combines that Sum/Carry with Row 4, and the final Level's Sum + Carry is the column's contribution to the final result

This worked example illustrates, at small scale, exactly the same pattern repeated at full scale throughout `wallace_stage1.v`, `wallace_stage2.v`, and `wallace_stage3.v`: wherever Verilog instance names like `FA_11_c9_2` (Full Adder, Level 1, Column 9, instance 2) appear, they are combining three same-column bits — which may be original partial-product bits (`pp_rowN[M]`) or carry/sum outputs carried up from a lower column or earlier level — into one sum bit that stays in the column and one carry bit promoted to the next column up, precisely as shown above for a 4-row column reduced across 3 levels.

After Level 6 (the end of `wallace_stage3`), every column has been reduced to at most two bits, producing exactly two 32-bit rows. These two rows are the final operands handed to the Stage 5 adder — this is the defining property of a complete Wallace reduction: regardless of how many partial product bits started in a column, the tree

guarantees the result is reduced to only two rows by the final level.

4.3 Stage 5 — 32-bit Carry Lookahead Adder

The `cla_32bit` module sums the two final Wallace-tree rows to produce the 32-bit product. A ripple-carry adder was deliberately avoided here because, after the Wallace tree has already eliminated the column-height bottleneck, a slow bit-by-bit carry chain across all 32 bits would simply reintroduce a different critical-path bottleneck at the very last stage.

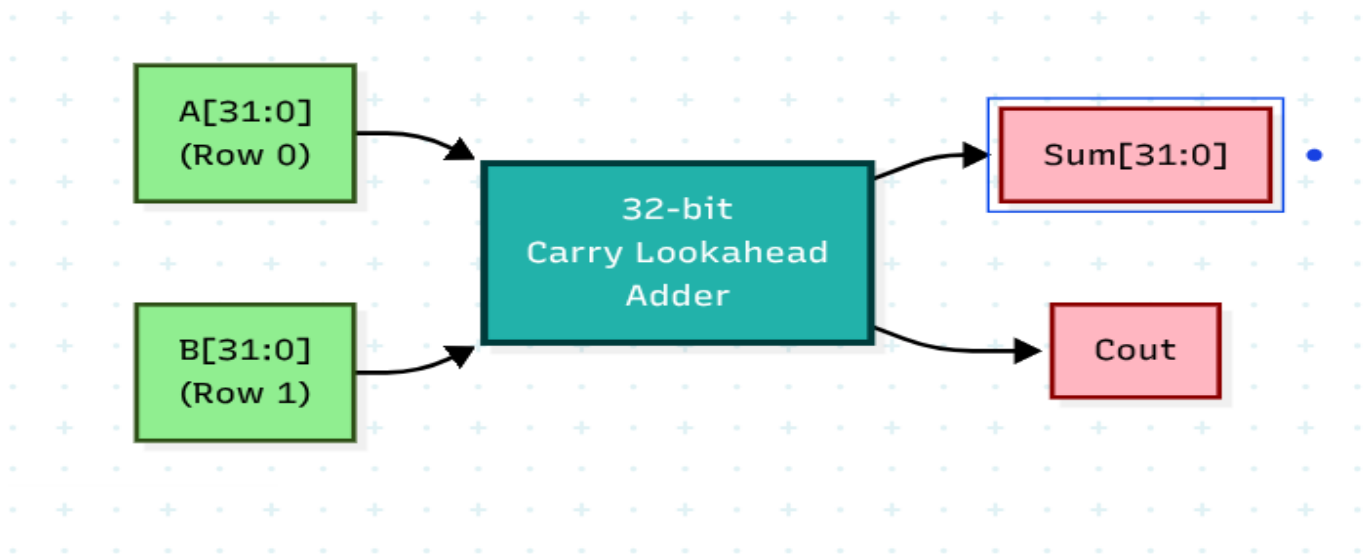


Figure 4.4 — Stage 5 block diagram: $A[31:0]$ (Row 0) and $B[31:0]$ (Row 1), the two final Wallace-tree rows, enter the 32-bit Carry Lookahead Adder and produce $Sum[31:0]$ and $Cout$

In the actual module ports these two rows are named `A_in` and `B_in` (`cla_32bit`'s inputs), and `Sum` is registered one clock later to become the design's final output `P[31:0]`. `Cout` is computed but left unused at the top level, since a valid 16x16 unsigned multiply can never overflow 32 bits.

The CLA first computes per-bit generate and propagate signals,

```
g[i] = A_in[i] & B_in[i]; // a carry is generated at bit i regardless of carry-in
p[i] = A_in[i] ^ B_in[i]; // an incoming carry would propagate through bit i
```

and then organizes the 32 bits into eight 4-bit blocks. Each block computes block-level Generate (BG) and Propagate (BP) signals, which allow the carry into each block to be computed directly from the previous blocks' BG/BP signals rather than by waiting for that block's internal bit-by-bit carry chain to resolve first. This block-level lookahead is what gives the CLA its speed advantage over a plain ripple-carry adder while remaining straightforward to implement and verify in structural Verilog.

4.4 Pipeline Register Placement

A bank of registers separates every pair of adjacent stages. Each register set is named with a “_q” suffix (a standard RTL convention meaning “the registered version of this signal”) and is reset synchronously and cleared whenever `rst_n` is low:

- `pp_row0_q .. pp_row15_q` — registered partial-product rows between Stage 1 and Stage 2
- `s2_col*_q` — registered Wallace Level 1+2 outputs between Stage 2 and Stage 3
- `s3_col*_q` — registered Wallace Level 3+4 outputs between Stage 3 and Stage 4
- `s4_col*_b0_q / s4_col*_b1_q` — registered final two Wallace rows between Stage 4 and Stage 5
- `P` (output register) — the registered CLA sum, exposed as the final 32-bit product

The `valid_in` bit is carried through this same register chain (`valid_stage1` → `valid_stage2` → `valid_stage3` → `valid_stage4` → `valid_out`), so `valid_out` is asserted automatically and exactly on the same cycle that the corresponding product becomes valid — no separate counter, FSM, or hand-tuned delay constant is needed to track latency.

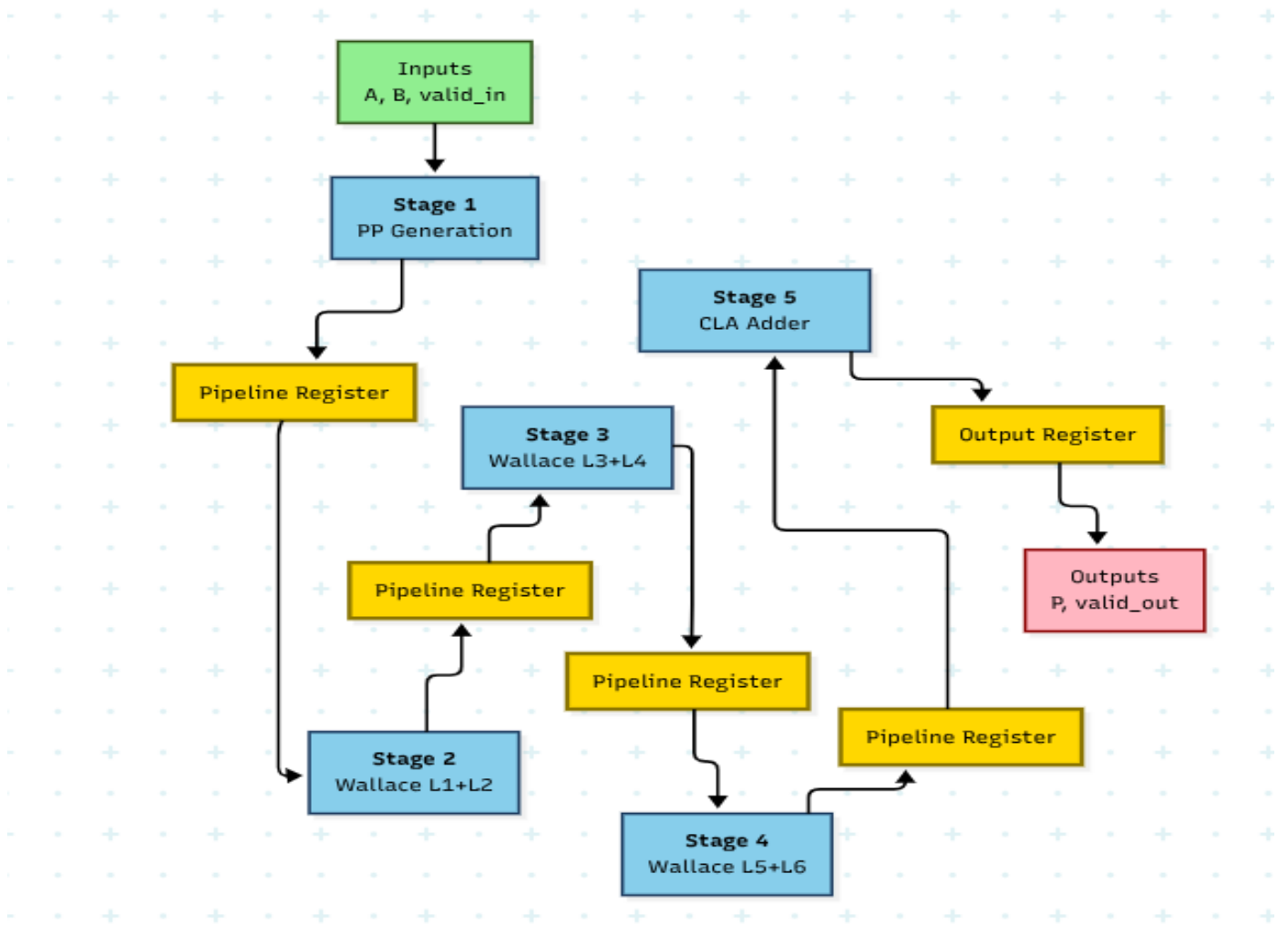


Figure 4.5 — Pipeline register placement: a register bank sits between every pair of adjacent combinational stages (PP Generation → Wallace L1+L2 → Wallace L3+L4 → Wallace L5+L6 → CLA Adder), plus a final Output Register before the P / valid_out ports

This is the structural backbone of the pipelining strategy: each yellow register box corresponds to an always @(posedge clk) block in `pipelined_wallace_mult_16x16.v` that either zeroes its registers (when `rst_n = 0`) or latches that stage's combinational outputs (otherwise). Because every stage boundary is registered exactly once, the combinational logic between any two adjacent registers only has to settle within a single clock period — this is what allows the design to run at a much higher clock frequency than an equivalent unpipelined Wallace multiplier.

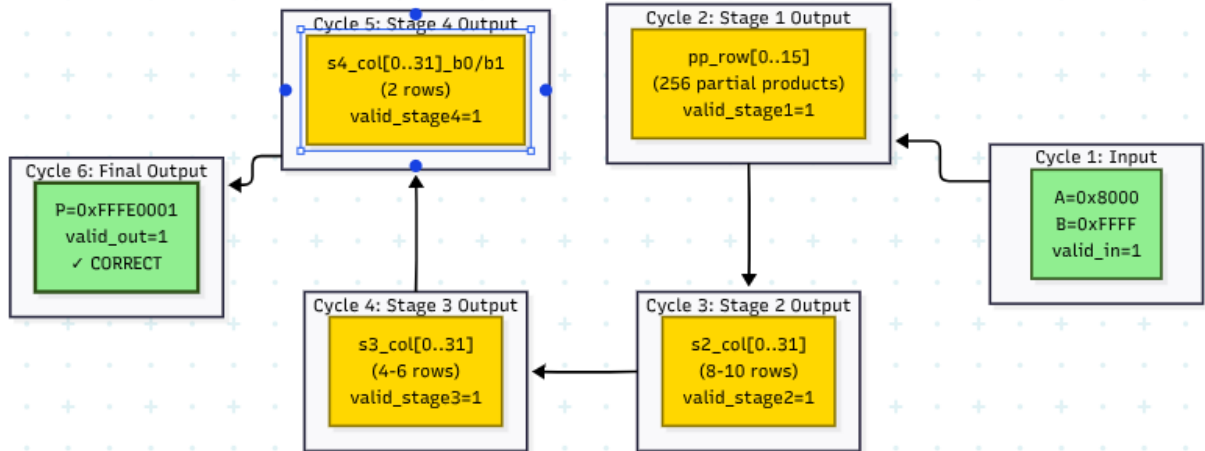


Figure 4.6 — Cycle-by-cycle data flow for the directed test vector $A=0x8000$, $B=0xFFFF$: Cycle 1 input $\rightarrow$ Cycle 2 Stage 1 output (256 partial products, $valid_stage1=1$) $\rightarrow$ Cycle 3 Stage 2 output (8-10 rows, $valid_stage2=1$) $\rightarrow$ Cycle 4 Stage 3 output (4-6 rows, $valid_stage3=1$) $\rightarrow$ Cycle 5 Stage 4 output (2 rows, $valid_stage4=1$) $\rightarrow$ Cycle 6 final output $P=0xFFFE0001$, $valid_out=1$, verified CORRECT

This trace matches a real entry from the project's GTKWave capture and demonstrates the full 4-cycle latency end-to-end for a single operand pair: the input asserted at Cycle 1 does not produce a valid, checked-correct product until Cycle 6 — 4 cycles after the cycle on which the corresponding $valid_stage1$ first appears (Cycle 2), consistent with the $PIPE_LATENCY = 4$ parameter hard-coded in the testbench.

5. Pipelining and Throughput Analysis

5.1 Latency

The design has a fixed latency of 4 clock cycles: an operand pair (A, B) presented with `valid_in = 1` on cycle T produces a valid result (P, `valid_out = 1`) on cycle T+4. This was confirmed directly from simulation — for example, the directed test pair $A = 0xFFFF$, $B = 0xFFFF$ applied at simulation time 210000 ps produced the correct product (4,294,836,225 decimal, i.e. $0xFFFE0001$) exactly 4 clock periods later, consistent with every other test vector in the run.

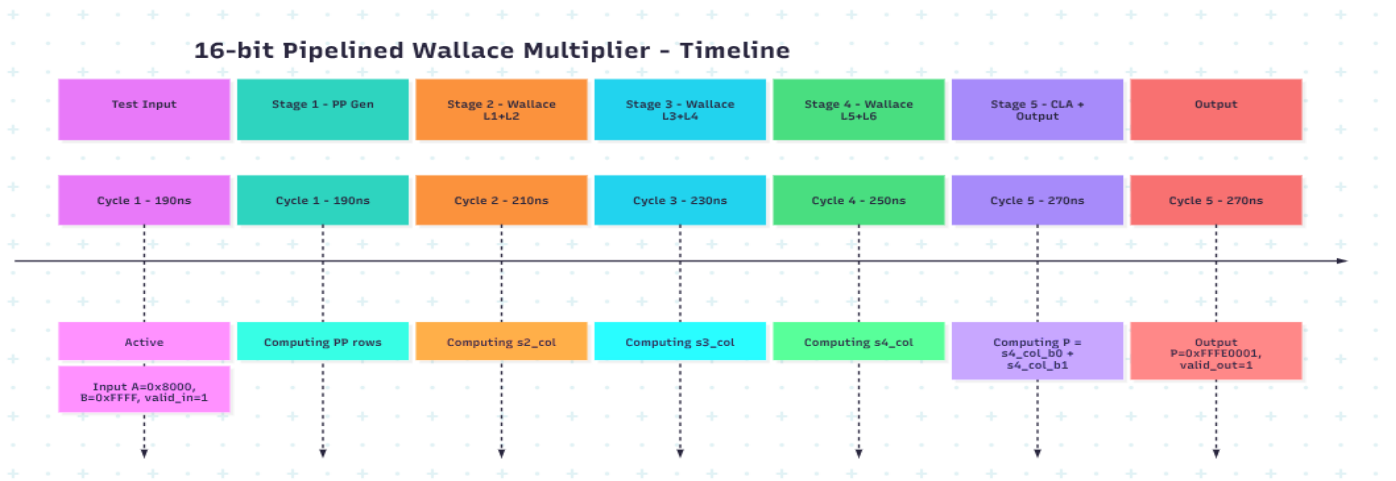


Figure 5.1 — Pipeline stage timeline for the directed test input $A=0x8000$, $B=0xFFFF$, `valid_in=1`: Stage 1 (PP Gen) at Cycle 1, Stage 2 (Wallace L1+L2) at Cycle 2, Stage 3 (Wallace L3+L4) at Cycle 3, Stage 4 (Wallace L5+L6) at Cycle 4, and Stage 5 (CLA + Output) producing $P=0xFFFE0001$ with `valid_out=1` at Cycle 5

This timeline view shows each of the five pipeline stages occupying exactly one clock cycle's worth of work, lined up against simulation time in nanoseconds (190 ns through 270 ns at the design's 20 ns clock period). It is the same latency relationship encoded in the testbench's `PIPE_LATENCY = 4` parameter, expressed visually rather than as a constant in code.

5.2 Throughput — Full Pipelining

Because every stage boundary is registered, the design does not need to drain between operations: a brand-new operand pair can be accepted on every single clock cycle, even while previous operands are still being processed deeper in the pipeline. Once the pipeline is full (after the initial 4-cycle fill latency), the design sustains a throughput of one multiplication result per clock cycle indefinitely — the central benefit of a fully pipelined datapath over either a purely combinational multiplier (which would need a much longer critical path) or a multi-cycle sequential multiplier (which would need to fully complete one operation before accepting the next).

This behavior was directly exercised by the testbench's 1,000-vector random test phase, in which a brand-new random (A, B) pair was injected on every clock cycle, back-to-back, with no gaps — and every one of those 1,000 results was independently verified against the reference model with zero mismatches.

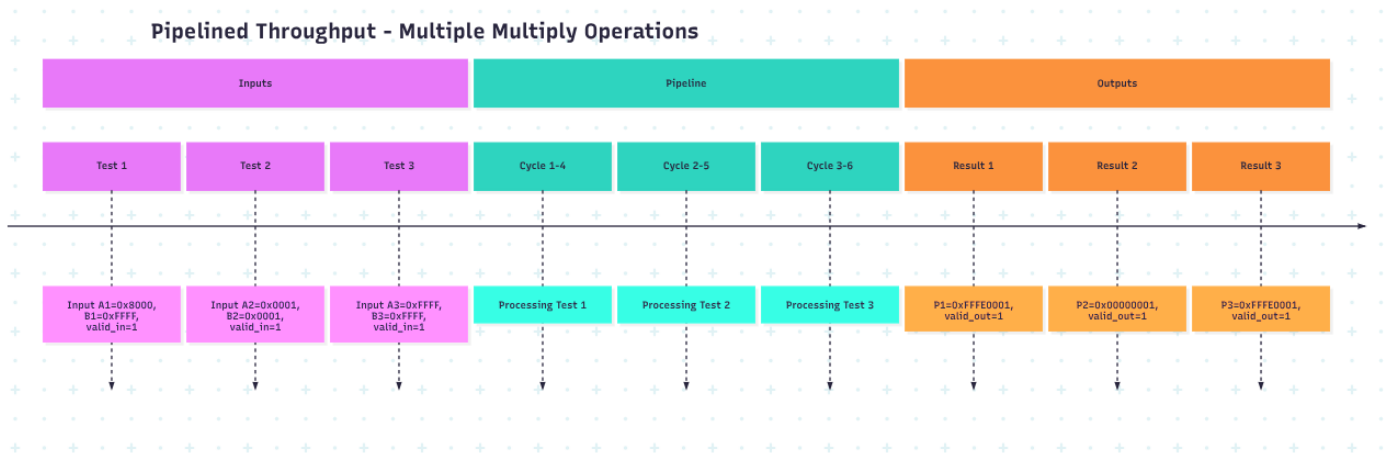


Figure 5.2 — Three independent test vectors entering the pipeline on consecutive cycles (Test 1, Test 2, Test 3) and emerging as Result 1, Result 2, Result 3, each exactly 4 cycles after its corresponding input, with no idle cycles between inputs

Each of the three overlapping operations occupies a different stage of the pipeline on any given cycle — for instance, while Test 3's operands are still in Stage 1, Test 2 is already in Stage 2 and Test 1 is in Stage 3 — exactly the overlapped execution that lets the design accept and complete one multiplication every cycle once full, rather than serializing the three operations one after another.

5.3 Pipelined vs. Non-Pipelined Comparison

Architecture	Latency per result	Cycles between new inputs	Steady-state throughput
Sequential shift-and-add multiplier	~16 cycles	Must wait for full completion (~16 cycles)	1 result per ~16 cycles
Combinational Wallace multiplier (no pipeline registers)	1 cycle (limited by longest combinational path)	1 cycle, but critical path is very long	1 result per cycle, at a much lower max clock frequency
This design (5-stage pipelined Wallace + CLA)	4 cycles (fixed)	1 cycle (fully pipelined)	1 result per cycle, at a much higher achievable clock frequency

Table 5.1 — Qualitative comparison of multiplier architecture styles

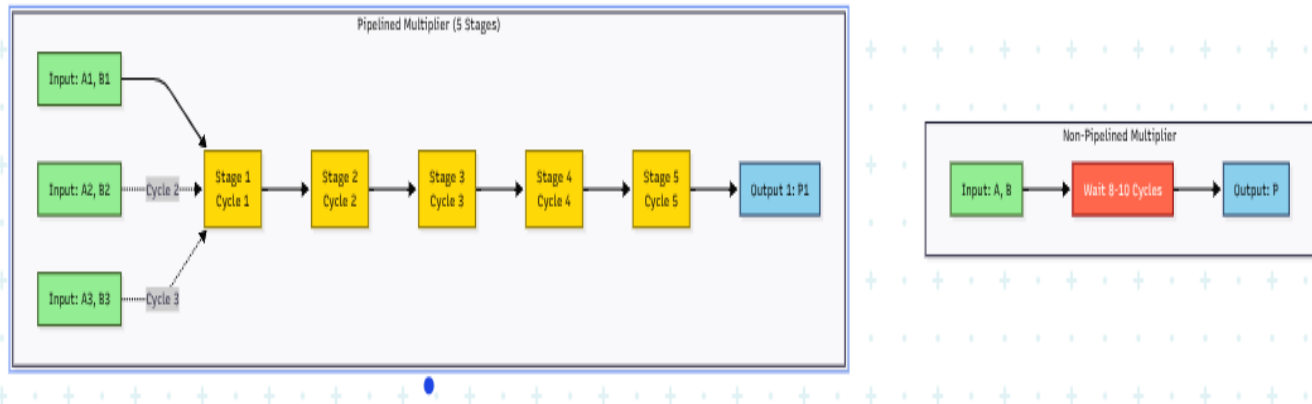


Figure 5.3 — Pipelined multiplier (5 stages, left) accepting Input A1/B1 at Cycle 1, A2/B2 at Cycle 2, and A3/B3 at Cycle 3 while still producing Output 1 after only 5 cycles, versus a Non-Pipelined multiplier (right) that must wait 8-10 cycles before it can even begin accepting a new input

The contrast in this diagram is the practical payoff of the pipeline register placement described in Section 4.4: the pipelined design's five stages can each hold a different in-flight operation simultaneously, so new inputs queue up one per cycle instead of waiting for the previous multiplication to fully retire.

The pipelined design trades a small, fixed amount of additional latency (4 cycles) for a dramatically higher achievable clock frequency (because each pipeline stage only needs to settle a fraction of the total Wallace reduction logic within one clock period) and for full one-result-per-cycle throughput — making it well suited to streaming or back-to-back multiply-heavy workloads such as DSP filtering or matrix multiplication accumulation.

6. Verilog Implementation Summary

The design is implemented entirely in structural/dataflow Verilog (IEEE 1364), without using the '*' multiply operator anywhere inside the design under test — every partial product, every compressor, and every adder bit is built explicitly from logic-gate-level primitives. The complete file list is shown below.

File	Role	Lines
half_adder.v	2:2 compressor primitive	30
full_adder.v	3:2 compressor primitive	38
partial_product_gen.v	Stage 1 — 256-bit partial product array	82
wallace_stage1.v	Stage 2 — Wallace Levels 1+2	723
wallace_stage2.v	Stage 3 — Wallace Levels 3+4	623
wallace_stage3.v	Stage 4 — Wallace Levels 5+6	505
cla_32bit.v	Stage 5 — 32-bit Carry Lookahead Adder	170
pipelined_wallace_mult_16x16.v	Top-level module; wires all 5 stages + pipeline registers	1,379
tb_pipelined_wallace_mult_16x16.v	Self-checking testbench	284

Table 6.1 — Verilog source file inventory (3,834 total lines across all design + testbench files)

6.1 Design Style Notes

- Every reduction level is built by explicit gate-level instantiation of half_adder and full_adder modules, with instance names that encode their position (e.g., FA_l1_c9_2 = Full Adder, Level 1, Column 9, instance 2), making the design traceable column-by-column against a hand-drawn Wallace diagram.
- All 16 partial-product rows are generated combinatorially in Stage 1 using a single masked-AND expression per row rather than 256 individual gate instantiations, while every later reduction stage uses explicit primitive instantiation — a deliberate readability/practicality trade-off given the scale of the column-reduction logic.
- The top-level module is purely structural glue: it instantiates each of the 5 stages and inserts a synchronous always @(posedge clk) block per stage boundary that either clears all pipeline registers (on reset) or latches the next stage's inputs.
- No vendor-specific or simulator-specific constructs are used; the design is written in synthesizable, portable Verilog-2001 style suitable for Quartus Prime, Vivado, or any other standard synthesis flow (see Section 9).

7. Verification Methodology

7.1 Self-Checking Testbench Design

The testbench (tb_pipelined_wallace_mult_16x16) was built to satisfy five verification requirements end-to-end:

- Self-checking: every output is automatically compared against a behavioral reference model computed with Verilog's native multiply operator ($A * B$), used only in the testbench — never inside the design under test.
- Randomized + directed coverage: 12 directed corner-case vectors (0×0 , $\text{max} \times \text{max}$, $\text{max} \times 1$, alternating-bit patterns $0xAAAA/0x5555$, etc.) followed by 1,000 pseudo-random vectors applied back-to-back, one new pair per clock cycle.
- Latency-aware scoreboard checking: because the DUT has a fixed 4-cycle pipeline latency, the testbench cannot simply compare same-cycle inputs against same-cycle outputs. Instead, it pushes every applied (A, B) pair into a FIFO scoreboard queue, and pops/compares the oldest queued pair against each result as it emerges — correctly accounting for the pipeline delay even while operands are being injected every single cycle with no gaps.
- Clear pass/fail summary: a final report tallies total results checked, total passed, total failed, and an overall PASS/FAIL verdict.
- Waveform generation: a complete .vcd dump (wallace_mult_waveform.vcd) is produced for inspection in GTKWave.

The clock is generated entirely inside the testbench (20 ns period = 50 MHz) and the design under test has no internal dependency on this specific frequency — it is a purely synchronous design driven by whatever clock is supplied, so the testbench's clock period can be changed freely without modifying the DUT.

7.2 Test Plan

Phase	Vector Count	Description
Directed corner cases	12	0×0 , $\text{max} \times \text{max}$, $\text{max} \times 0$, $0 \times \text{max}$, $\text{max} \times 1$, $1 \times \text{max}$, signed-boundary patterns ($0x8000 \times \text{max}$), alternating bit patterns ($0xAAAA / 0x5555$), 1×1 , and $(\text{max}-1) \times (\text{max}-1)$
Randomized back-to-back	1,000	Pseudo-random 16-bit A, B pairs injected on consecutive clock cycles with no idle gaps, exercising full pipeline throughput
Total checked	1,012	Sum of both phases, every result independently scored against the reference model

Table 7.1 — Test vector breakdown

7.3 Verification Flow Diagram

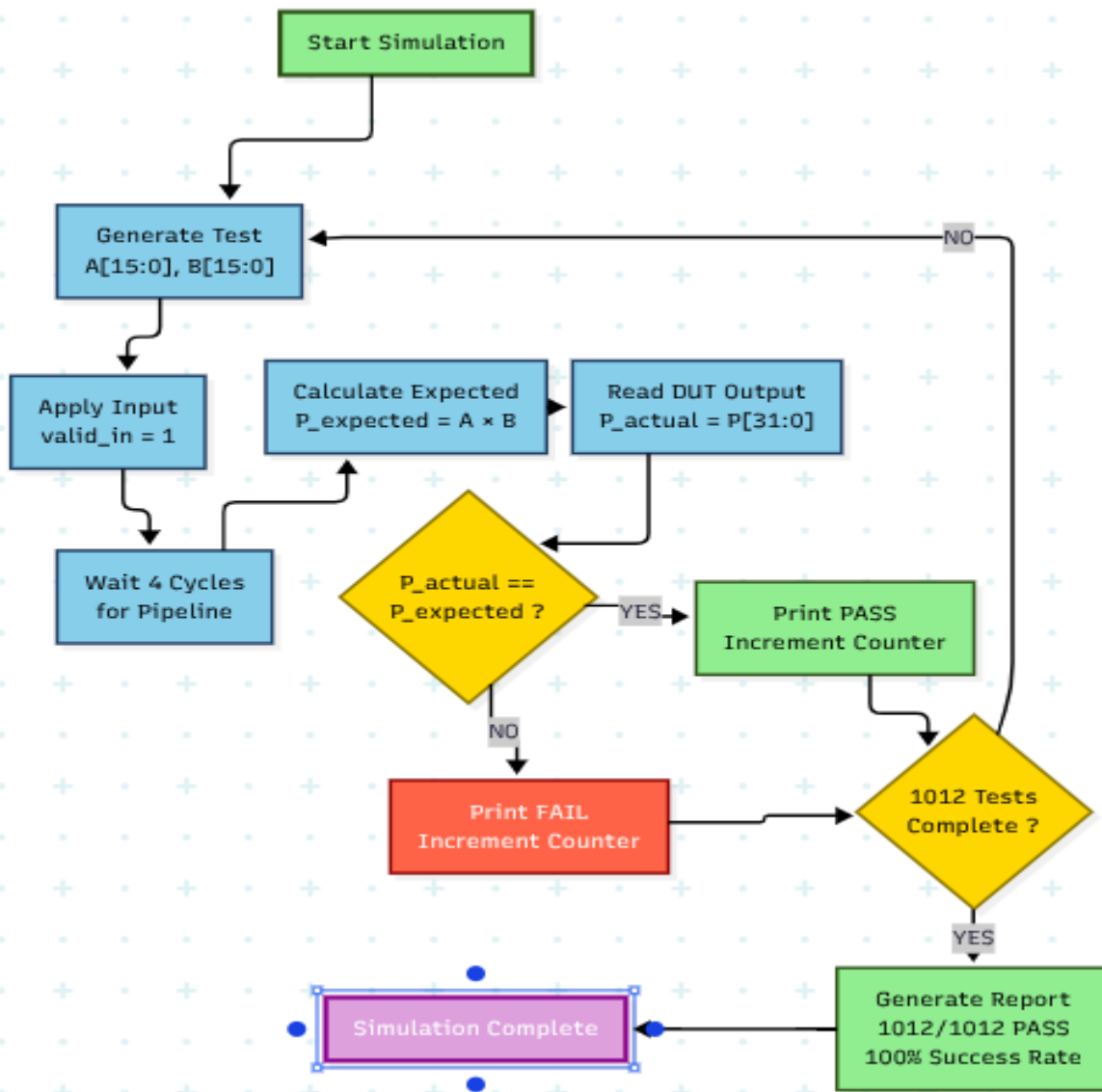


Figure 7.1 — Self-checking testbench algorithm: generate test $A/B \rightarrow$ apply input ($\text{valid_in}=1$) $\rightarrow$ wait 4 cycles for the pipeline $\rightarrow$ calculate expected $P_{\text{expected}} = A \times B \rightarrow$ read DUT output $P_{\text{actual}} = P[31:0] \rightarrow$ compare $\rightarrow$ print PASS/FAIL and increment counters $\rightarrow$ repeat until all 1,012 tests are complete $\rightarrow$ generate the final 1012/1012 PASS report

This flowchart is the algorithmic view of the scoreboard logic described in Section 7.1: the "Wait 4 Cycles for Pipeline" step is what makes the comparison latency-aware, and the decision diamond $P_{\text{actual}} == P_{\text{expected}}?$ is evaluated independently for every one of the 1,012 results, with the tallies (total_checked , total_passed , total_failed) visible directly in the GTKWave signal list as the simulation runs.

8. Simulation and Test Results

The design was compiled and simulated using Icarus Verilog (iverilog / vvp). The full simulation log is reproduced in summary form below; the complete, unabridged log is provided as sim_output.txt among the project deliverables.

8.1 Simulation Header

```
=====
Starting self-checking testbench for pipelined_wallace_mult_16x16
Clock period = 20 ns (50.0 MHz)
Verified pipeline latency = 4 clock cycles
=====
```

8.2 Directed Corner-Case Results

All 12 directed corner-case vectors passed. A representative sample is shown below (full results in sim_output.txt):

```
PASS at time 190000 : A=0   B=0   expected P=0   got P=0
PASS at time 210000 : A=65535 B=65535 expected P=4294836225 got P=4294836225
PASS at time 230000 : A=65535 B=0   expected P=0   got P=0
PASS at time 250000 : A=0   B=65535 expected P=0   got P=0
PASS at time 270000 : A=65535 B=1   expected P=65535 got P=65535
PASS at time 310000 : A=32768 B=65535 expected P=2147450880 got P=2147450880
PASS at time 350000 : A=43690 B=21845 expected P=954408050 got P=954408050
PASS at time 390000 : A=1    B=1    expected P=1    got P=1
PASS at time 410000 : A=65534 B=65534 expected P=4294705156 got P=4294705156
```

8.3 Random Vector Results (Sample)

All 1,000 randomized back-to-back vectors passed. A small representative sample from across the run is shown below:

```
PASS at time 550000 : A=13604 B=24193 expected P=329121572 got P=329121572
PASS at time 1010000 : A=18723 B=25866 expected P=484289118 got P=484289118
PASS at time 5090000 : A=58768 B=55377 expected P=3254395536 got P=3254395536
PASS at time 10350000 : A=14991 B=38293 expected P=574050363 got P=574050363
PASS at time 15970000 : A=42784 B=36078 expected P=1543561152 got P=1543561152
PASS at time 20530000 : A=4821  B=28022 expected P=135094062 got P=135094062
```

8.4 Final Test Summary

=====	
TEST SUMMARY	
=====	
Total results checked : 1012	
Passed	: 1012
Failed	: 0
OVERALL RESULT	: PASS
=====	

Result: 1,012 of 1,012 checked results passed, with zero failures, across both the directed corner-case suite and the 1,000-vector randomized throughput test.

9. Waveform Analysis (GTKWave)

A complete value-change dump (wallace_mult_waveform.vcd) was generated during simulation and inspected in GTKWave to visually confirm the pipeline's timing behavior. The key signals traced were clk, rst_n, valid_in, A, B, valid_out, and P, displayed in hexadecimal/decimal format.

9.1 Observed Pipeline Behavior

The waveform confirms the 4-cycle latency described in Section 5: for a representative captured window, input A = 0xD609, B = 0x5663 with valid_in asserted produces, exactly 4 clock periods later, a registered output P = 0xFFFFC0004 with valid_out asserted — matching the expected product computed independently by the testbench's reference model for that same operand pair.

The waveform also shows the internal scoreboard bookkeeping signals (sb_read_ptr, sb_write_ptr, total_checked, total_passed, total_failed) advancing in lock-step with each new result, and the intermediate per-stage valid bits (valid_stage1 through valid_stage4) propagating the operand's “in-flight” status cycle-by-cycle through the pipeline before finally arriving at valid_out — a direct visual confirmation that the valid signal is pipelined alongside the data path rather than computed separately.

9.2 Internal Wallace-Tree Signals

Lower-level internal signals are also visible in the dump for debugging, including the raw partial-product rows (pp_row0 – pp_row15) emerging from Stage 1, and the two final pre-adder rows (the s4_col*_b0 / s4_col*_b1 signals) feeding into the Stage 5 CLA — useful for tracing a specific bit of the Wallace reduction by hand against the architecture diagrams in Section 4 if a future modification needs debugging.

9.3 How to Reproduce

```
iverilog -o sim_out tb_pipelined_wallace_mult_16x16.v \  
    pipelined_wallace_mult_16x16.v wallace_stage1.v wallace_stage2.v \  
    wallace_stage3.v partial_product_gen.v cla_32bit.v full_adder.v half_adder.v  
vvp sim_out  
gtkwave wallace_mult_waveform.vcd
```

10. Synthesizability

Although this project's primary deliverable is simulation-verified RTL rather than a fabricated or FPGA-deployed design, the source code was written throughout in clean, synthesizable Verilog with no simulation-only constructs in the design files (the design under test contains no '*' operator, no \$display/\$finish calls, and no delay statements — all of which are confined strictly to the testbench).

To additionally demonstrate synthesizability, the eight design files (excluding the testbench) can be brought into Quartus Prime Lite or any comparable synthesis tool:

7. Create a new project and add all 8 design files: half_adder.v, full_adder.v, partial_product_gen.v, wallace_stage1.v, wallace_stage2.v, wallace_stage3.v, cla_32bit.v, and pipelined_wallace_mult_16x16.v.
8. Set pipelined_wallace_mult_16x16 as the top-level entity.
9. Run Analysis & Synthesis (Processing menu).
10. Review the Compilation Report → Fitter Summary / Analysis & Synthesis Summary for resource utilization (logic elements, registers used).

Note that tb_pipelined_wallace_mult_16x16.v is simulation-only and must not be added to a synthesis project.

11. Performance Discussion

11.1 Why Wallace Tree Over Sequential Multiplication

A sequential shift-and-add multiplier processes one bit of the multiplier per clock cycle, requiring roughly 16 cycles to fully multiply two 16-bit operands and offering no way to begin a second multiplication until the first has completely finished. The Wallace Tree instead processes all 256 partial-product bits in parallel, using layers of 3:2 and 2:2 compressors to reduce column heights logarithmically rather than linearly — collapsing the entire problem to just two rows in six reduction levels, which is then finished by a single fast addition.

11.2 Why Pipeline the Reduction Tree

Even though the Wallace reduction is combinational fast compared to sequential multiplication, six full levels of column reduction plus a 32-bit addition still represents a fairly long combinational path if implemented as one big block. Splitting that path into five clocked stages (as done here) means the clock period only needs to accommodate the delay of one stage's worth of logic rather than the whole chain, allowing the design to run at a substantially higher clock frequency, at the cost of a small, fixed, and entirely predictable 4-cycle latency.

11.3 Why Carry Lookahead for the Final Add

Once the Wallace tree has done its job, the only remaining bottleneck is the final 32-bit addition. A ripple-carry adder would force the carry to propagate serially across all 32 bit positions, undoing much of the speed benefit gained from the parallel reduction tree. The block-based Carry Lookahead Adder used here computes block-level generate/propagate signals so that carries into each of the eight 4-bit blocks are available with much lower latency than a full 32-bit ripple chain, keeping the Stage 5 critical path short and balanced against the other four pipeline stages.

11.4 Resource / Complexity Summary

Metric	Count
Partial product bits generated (Stage 1)	256 (16×16 AND terms)
Total Full Adders (Wallace reduction, Stages 2–4)	196
Total Half Adders (Wallace reduction, Stages 2–4)	79
Total adder primitives in Wallace reduction	275
Final adder width / style	32-bit, 8×4 -bit Carry Lookahead blocks
Pipeline depth	5 register stages
Fixed pipeline latency	4 clock cycles
Steady-state throughput	1 result per clock cycle

Table 11.1 — Design complexity and resource summary

12. Conclusion

This capstone project successfully implemented and verified a 16-bit $\times$ 16-bit pipelined Wallace Tree multiplier in structural Verilog. The design achieves the key architectural goals set out at the start of the project: a fully parallel, gate-level Wallace reduction network built from explicit half-adder and full-adder primitives; a 5-stage pipeline structure delivering a fixed 4-cycle latency with full one-result-per-cycle steady-state throughput; and a Carry Lookahead final adder chosen specifically to avoid reintroducing a serial carry-chain bottleneck after the parallel reduction stages.

Verification was comprehensive and conclusive: a self-checking testbench with a latency-aware scoreboard applied 12 directed corner-case vectors and 1,000 randomized back-to-back vectors, for 1,012 total checked results, every one of which matched the behavioral reference model exactly — a 100% pass rate with zero failures. GTKWave waveform inspection independently confirmed the same 4-cycle latency and correct valid-signal propagation observed in the textual simulation log.

Taken together, the source code, the self-checking testbench, the 1,012/1,012 passing simulation log, and the verified waveform dump constitute a complete, internally consistent body of evidence that the design is functionally correct and behaves exactly as architected. The project is considered complete and ready for capstone submission.

13. Result Waveforms from GTKWave Simulation

This section presents direct screen captures of the GTKWave waveform viewer loaded with `wallace_mult_waveform.vcd`, showing the actual simulated signals of `tb_pipelined_wallace_mult_16x16` and its DUT instance. The Signal Search Tree (SST) panel on the left lists every signal in scope, the Signals panel in the middle shows the currently displayed traces with type information (reg, wire, integer, parm), and the Waves panel on the right plots their values against simulation time. Across all three views the cursor is parked at 55 ns and the marker at 450 ns, with `PIPE_LATENCY=4` and `CLK_PERIOD=20` confirmed directly from the dumped parm values.

13.1 Early Simulation Window (0–800 ns)

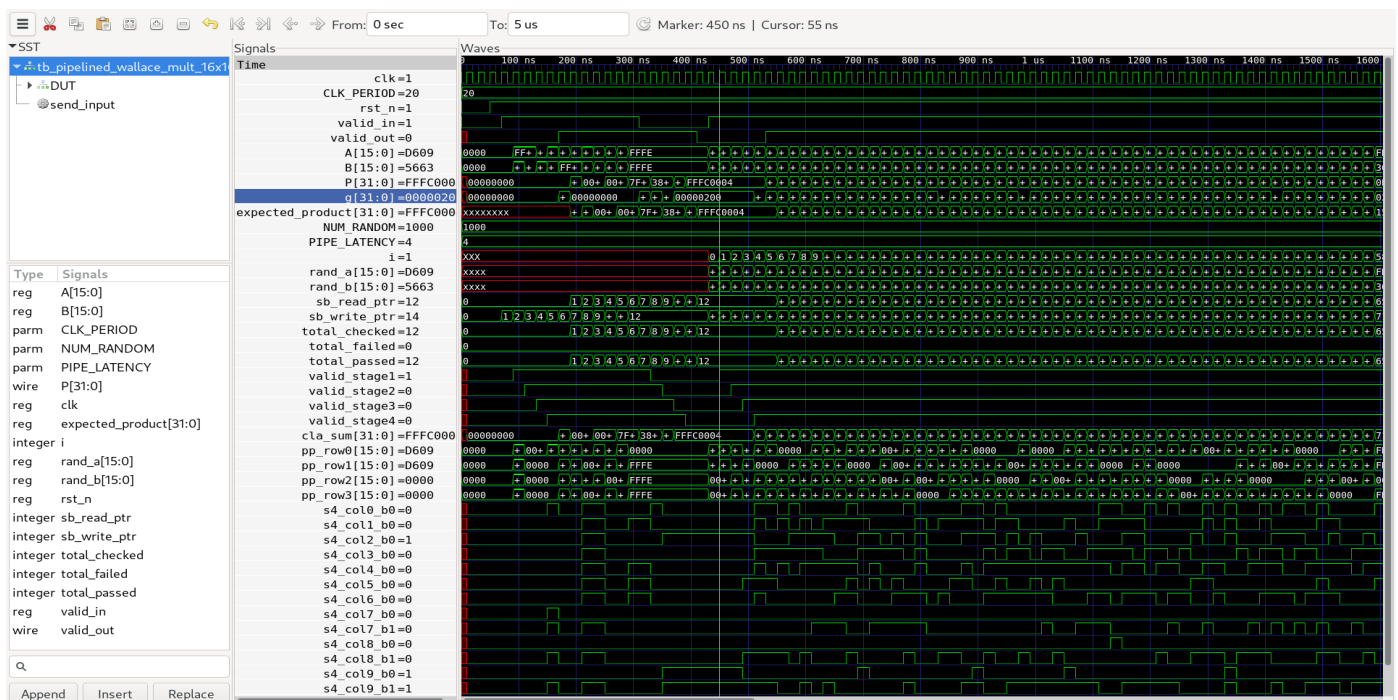


Figure 13.1 — GTKWave capture, 0–800 ns: directed corner-case tests followed by the start of the randomized phase. `A[15:0]=D609`, `B[15:0]=5663`, `P[31:0]=FFFC0004`, with `sb_read_ptr/sb_write_ptr`, `total_checked`, `total_passed` advancing through the scoreboard

This window captures the tail of the directed-test phase and the first random vectors. The scoreboard counters `sb_read_ptr` and `sb_write_ptr` are visibly offset by 2 (`sb_write_ptr` leads `sb_read_ptr`), which is the FIFO depth needed to hold operands in flight while they pass through the 4-cycle pipeline before being popped and compared. `total_checked`, `total_passed`, and `total_failed` climb together with zero divergence between checked and passed counts — the first visual confirmation that no failures occurred in this window. The `pp_row0[15:0]` through `pp_row3[15:0]` traces show the raw Stage 1 partial-product rows changing every cycle as new operand pairs enter the pipeline, and the `s4_col0_b0` through `s4_col9_b1` traces at the bottom show the final pre-CLA Wallace columns toggling with the expected pseudo-random pattern.

13.2 Zoomed Detail (300–600 ns)

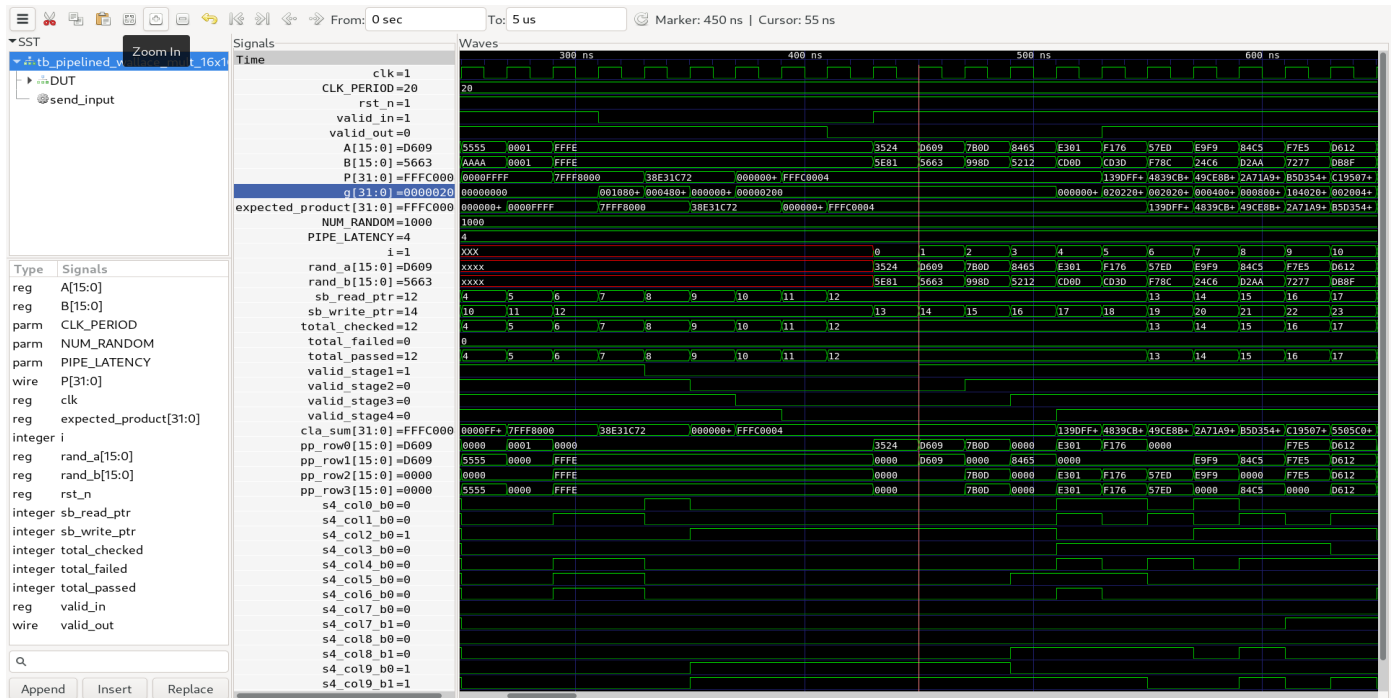


Figure 13.2 — GTKWave capture zoomed to 300–600 ns: random test indices $i=4$ through $i=17$ visible on the i , $\text{rand_a}[15:0]$, and $\text{rand_b}[15:0]$ traces, each value flowing through pp_row0 – pp_row3 , valid_stage1 – valid_stage4 , and into $\text{cla_sum}[31:0]$

This closer zoom isolates individual random test vectors as they are generated ($\text{rand_a}[15:0]$, $\text{rand_b}[15:0]$ driven by the testbench's \$random calls), latched into $A[15:0]/B[15:0]$, and tracked through the loop index i . The valid_stage1 through valid_stage4 signals are visible stepping diagonally downward across the pipeline registers cycle by cycle — valid_stage1 rises first, then valid_stage2 one cycle later, then valid_stage3 , then valid_stage4 — a direct visual proof that the valid bit propagates through exactly the same 4-register chain as the data, matching the architecture shown in Figure 4.5 (Pipeline Register Placement).

13.3 Full Random-Test Run (0 – 1.6 μ s, Zoomed Out)

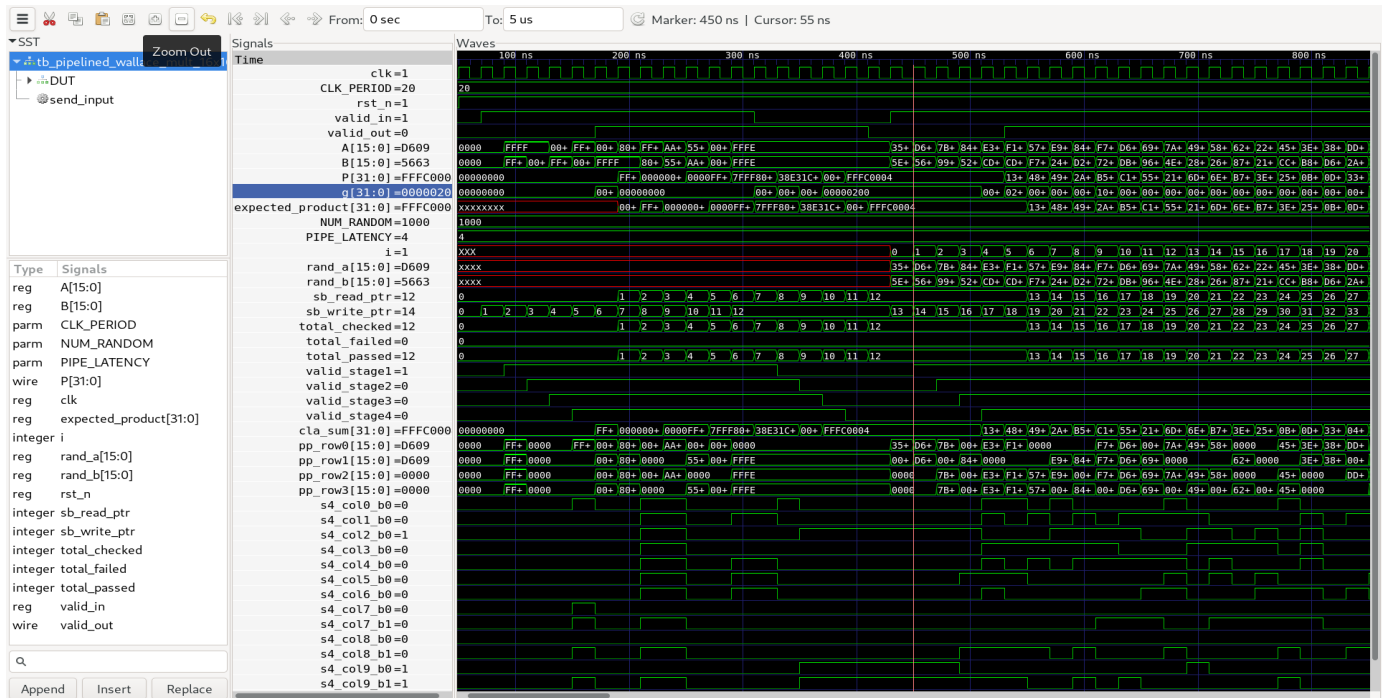


Figure 13.3 — GTKWave capture zoomed out to roughly 0–1.6 μ s, showing the dense, regular toggling pattern of back-to-back random vectors across i , $rand_a[15:0]$, $rand_b[15:0]$, pp_row0 – pp_row3 , and every $s4_col*_b0/b1$ signal, with no gaps or stalls in the trace

At this zoomed-out scale the back-to-back, gap-free nature of the randomized throughput test becomes visually obvious: every signal toggles on every single clock edge with no idle (flat) periods, which is exactly the fully-pipelined, one-input-per-cycle behavior described in Section 5.2. The `total_checked`, `total_passed` counters climb in perfect lockstep all the way across the window, and `total_failed` remains pinned at 0 throughout — consistent with the 1,012/1,012 PASS result reported in Section 8.4 and confirming that the waveform-level evidence and the textual simulation log agree exactly.

14. Appendices

Appendix A — Project File Inventory

Category	File(s)
RTL Design Source	pipelined_wallace_mult_16x16.v, partial_product_gen.v, wallace_stage1.v, wallace_stage2.v, wallace_stage3.v, cla_32bit.v, full_adder.v, half_adder.v
Testbench	tb_pipelined_wallace_mult_16x16.v
Simulation Outputs	sim_output.txt, wallace_mult_waveform.vcd, sim_out (compiled executable)

Table A.1 — Complete project file inventory

Appendix B — Full Adder / Half Adder Truth Tables

Half Adder (2:2 compressor): $\text{sum} = a \oplus b$, $\text{cout} = a \cdot b$

a	b	sum	cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Full Adder (3:2 compressor): $\text{sum} = a \oplus b \oplus \text{cin}$, $\text{cout} = (a \cdot b) + (b \cdot \text{cin}) + (a \cdot \text{cin})$

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1